

Diseño de un acelerador CORDIC

12/junio/2026

Isaac Hernández

isaac.carrazco@cinvestav.mx

Descripción del Problema

Diseñar un IP-core que realice el algoritmo CORDIC de rotación y vectorización sobre un vector aleatorio de norma 1 (vectores con ángulos aleatorios de $-\pi$ a π , o de 180° a 180°), dando un valor de entrada al IP-core θ , el cual realizará la operación de rotación del vector de tal ángulo, manteniendo la norma igual a 1.

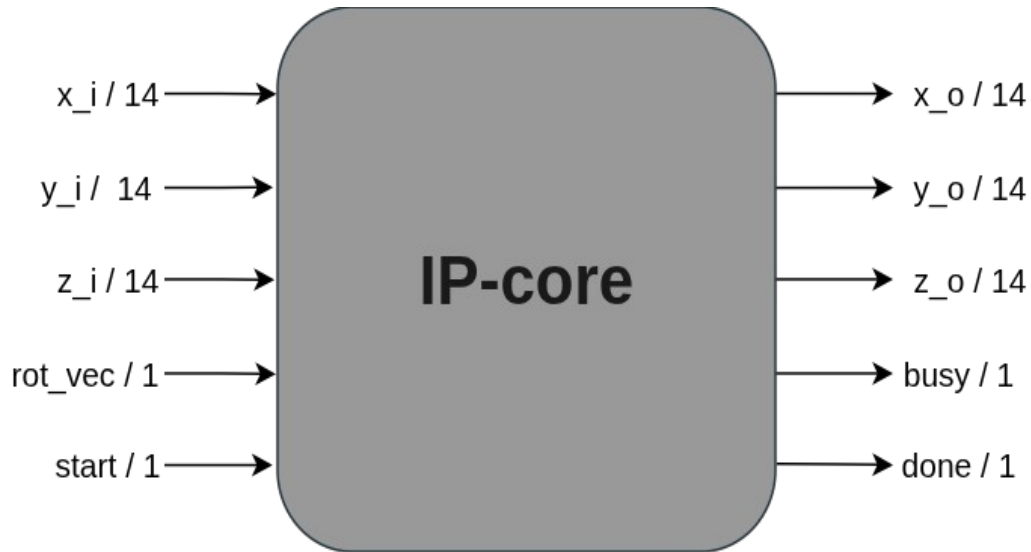
En el **modo de rotación**, se rota iterativamente el vector (x_i, y_i) usando el ángulo z_i como objetivo. En cada paso, se aplica una rotación que reduce z_i hasta que se aproxima a cero, logrando la rotación deseada.

En el **modo de vectorización**, se rota iterativamente el vector (x_i, y_i) hasta anular su componente y . El ángulo total acumulado en z corresponde al argumento del vector original, mientras que la componente x final es su magnitud.

Requerimientos

- El IP-core deberá realizar las rotaciones considerando valores negativos y positivos que estén en el rango -1 a 1 , considerando un ancho de palabra de hasta 32 bits.
- Diseño paramétrico.
- Generación de un modelo de oro.
- Análisis de punto fijo del algoritmo con $\text{SQNR} > 50$ dB.
- Módulo Top con diagrama de señales de entrada y salida conveniente y de libre elección.
- Implementar la metodología Top-Down.
- Implementar el coprocesador en HDL.
- Evaluación de métrica de desempeño de salida del IP-core con $\text{SQNR} > 50$ dB en simulación.

Entradas y Salidas



Entradas

- $x_i / 14$ es el valor x del vector de entrada.
- $y_i / 14$ es el valor y del vector de entrada.
- $z_i / 14$ es el ángulo de rotación final que se aplicará al vector de entrada.
- $rot_vec / 1$ indica el modo de operación del algoritmo CORDIC: 1 – rotación, 0 – vectorización.
- **start** / 1 inicia la operación con las entradas actuales.

Salidas

- $x_o / 14$ es el valor x del vector de salida.
- $y_o / 14$ es el valor y del vector de salida.
- $z_o / 14$ es el ángulo normalizado final acumulado por el algoritmo.
- **busy** / 1 indica actividad mientras se realiza la operación en el IP-core.
- **done** / 1 es una señal tipo One Shot que indica cuando la operación haya terminado.

Pseudocódigo

```
function [x_R, y_R, z_R] = cordic(x_in, y_in, theta, n, mode)
```

```
atan_table = atan(2.^-(0:n-1));
```

```
x_curr = x_in;
```

```
y_curr = y_in;
```

```

% Pre rotacion
if mode_vect
    % Vectorizacion
    if x_curr == 0 && y_curr == 0
        error("Operación inválida en modo vectorización: x = y = 0\n");
    end

    if x_curr < 0

        if y_curr > 0
            rot_ini = pi;
        else
            rot_ini = -pi;
        end

        x_curr = -x_curr;
        y_curr = -y_curr;

    else
        rot_ini = 0;
    end

    z_curr = 0;

else
    % Rotacion
    if theta >= pi/2
        z_curr = theta - pi/2;
        rot_ini = pi/2;
    elseif theta <= -pi/2
        z_curr = theta + pi/2;
        rot_ini = -pi/2;
    else
        z_curr = theta;
        rot_ini = 0;
    end

end

for i = 1:n

    % Asignar sigma(i)
    if (mode_rot && z_curr >= 0) || ...
        (mode_vect && y_curr < 0)
        sigma = 1;
    else
        sigma = -1;
    end

    % Calcular el micro rotamiento
    % El algoritmo contempla la primera iteracion con i = 0 ,pero matlab
    % trabaja con indices base 1, es por eso que se usa 2^(-i+1)
    x_shift = bitsra(x_curr, i-1);
    y_shift = bitsra(y_curr, i-1);

    x_nxt = x_curr - sigma * y_shift;
    y_nxt = y_curr + sigma * x_shift;
    z_nxt = z_curr - sigma * atan_table(i);

```

```

    x_curr = x_nxt;
    y_curr = y_nxt;
    z_curr = z_nxt;
end

z_R = z_nxt;

% Calcular rotaciones iniciales
if mode_rot && rot_ini == pi/2
    % Rotar +90° (x,y) => (-y,x)
    x_R = -y_nxt;
    y_R = x_nxt;
elseif mode_rot && rot_ini == -pi/2
    % Rotar -90° (x,y) => (y,-x)
    x_R = y_nxt;
    y_R = -x_nxt;
else
    x_R = x_nxt;
    y_R = y_nxt;
end

if mode_vect && rot_ini ~= 0
    z_R = z_R + rot_ini;
end

% Factor de escalamiento
k = 0.607252935;
x_R = x_R * k;
y_R = y_R * k;

end

```

Este algoritmo se puede dividir en 3 secciones principales:

1. Asignar valores iniciales para x, y, z, dependiendo de si hace falta una pre-rotación.
2. Realizar las iteraciones de los microrrotamientos.
3. Asignar los valores de salida con los obtenidos por el paso anterior, tomando en cuenta si hubo pre-rotación y el factor de escalamiento.

La **pre-rotación** es necesaria porque el radio de convergencia del algoritmo CORDIC se limita a ángulos en el intervalo $(-\pi/2, \pi/2)$. Para extender la cobertura al rango completo $(-\pi, \pi)$, se aplica una rotación inicial de $\pm 90^\circ$ (en modo rotación) o de $\pm 180^\circ$ (en modo vectorización cuando $x < 0$) antes de iniciar las iteraciones, y se compensa al final sumando dicha rotación al resultado acumulado en z.

Validación del pseudocódigo

Para validar la correctitud del pseudocódigo, se creó un script que compara el resultado de la función cordic previamente definida contra el resultado teórico calculado matemáticamente; explícitamente:

1. Modo rotación:
 - $x_R = x \cdot \cos(\theta) - y \cdot \sin(\theta)$

- $y_R = x \cdot \sin(\theta) + y \cdot \cos(\theta)$
- $z_R = 0$

2. Modo vectorización:

- $x_R = \text{norm}(x_{in}, y_{in})$
- $y_R = 0$
- $z_R = \text{atan2}(y_{in}, x_{in})$

Se consideró $n = 12$ como el número de iteraciones del algoritmo, con una tolerancia de $1e-3$. Cualquier error mayor que la tolerancia es tomado como una falla.

Los vectores de prueba para el **modo rotación** son los siguientes:

```
vectors_rot = [1, 0;    % eje x
               0, 1;    % eje y
               1, 1;    % diagonal
               -1, 0;   % cuadrante II / III (negativo en x)
               0, -1;   % eje -y
               0.5, 0.25]; % vector general

thetas = [-pi+0.01, -3*pi/4, -pi/2, -pi/4, -pi/8, ...
           0, pi/8, pi/4, pi/2, 3*pi/4, pi-0.01];
```

Los vectores de prueba para el **modo vectorización** son los siguientes:

```
% Puntos de prueba: cubrir los cuatro cuadrantes y casos límite
points_vect = [ 1, 0;    % eje +x
               0, 1;    % eje +y
               -1, 0;   % eje -x (pre-rotación 180°)
               0, -1;   % eje -y
               1, 1;    % Q1
               -1, 1;   % Q2
               -1, -1;  % Q3
               1, -1;   % Q4
               0.5, 0.25; % Q1 general
               -0.3, 0.7; % Q2 general
               -0.8, -0.6; % Q3 general
               0.6, -0.8]; % Q4 general
```

Análisis de punto fijo

Para determinar el formato de punto fijo adecuado, se desarrolló el script *cordic_test.m*, el cual itera sobre distintas configuraciones $Q_m.n$ y evalúa el SQNR resultante en cada una, comparando las salidas del algoritmo en punto fijo contra las del modelo de referencia en punto flotante de doble precisión.

Los valores de x e y se encuentran acotados en $[-1, 1]$. Sin embargo, durante las iteraciones del algoritmo CORDIC, los valores internos se ven escalados por el factor de ganancia $K \approx 1.6468$, por lo que los valores intermedios de x e y pueden alcanzar magnitudes de hasta ~ 1.65 antes de la corrección

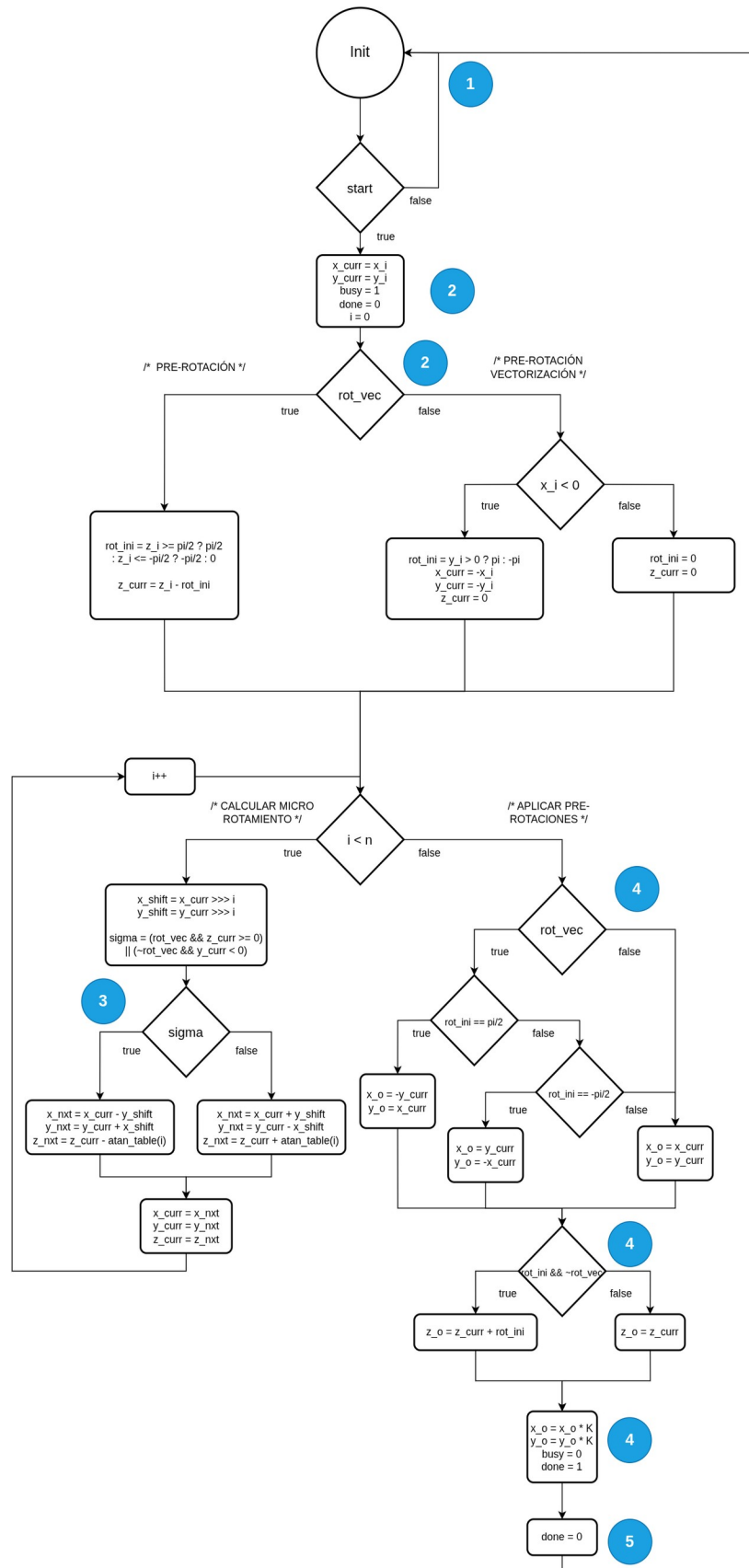
final. El ángulo z opera en el rango $[-\pi, \pi] \approx [-3.1416, 3.1416]$. El formato **Q3.10** proporciona un rango de representación de $[-4, 4)$, lo cual cubre todos estos casos sin producir desbordamiento.

Con 10 bits fraccionarios, el paso de cuantización es $\Delta = 2^{-10} \approx 9.77 \times 10^{-4}$. El SQNR teórico esperado para una señal de rango completo con cuantización uniforme sigue la relación aproximada:

$$SQNR \approx 6.02 \cdot N_f + 1.76 \text{ dB}$$

donde N_f es el número de bits fraccionarios. Para $N_f = 10$, se obtiene $SQNR \approx \mathbf{61.96 \text{ dB}}$, superando el umbral de 50 dB requerido. El script *cordic_test.m* confirmó empíricamente este resultado, evaluando el SQNR sobre un conjunto representativo de vectores de prueba en todos los cuadrantes y casos límite. Formatos con menos bits fraccionarios (p. ej. Q3.8) producían SQNR por debajo del umbral; formatos con más bits (p. ej. Q3.12) cumplen con el criterio del mínimo SQNR aceptable, pero requerían de tamaños de palabra mayores, que se traducen a un consumo de recursos mayor.

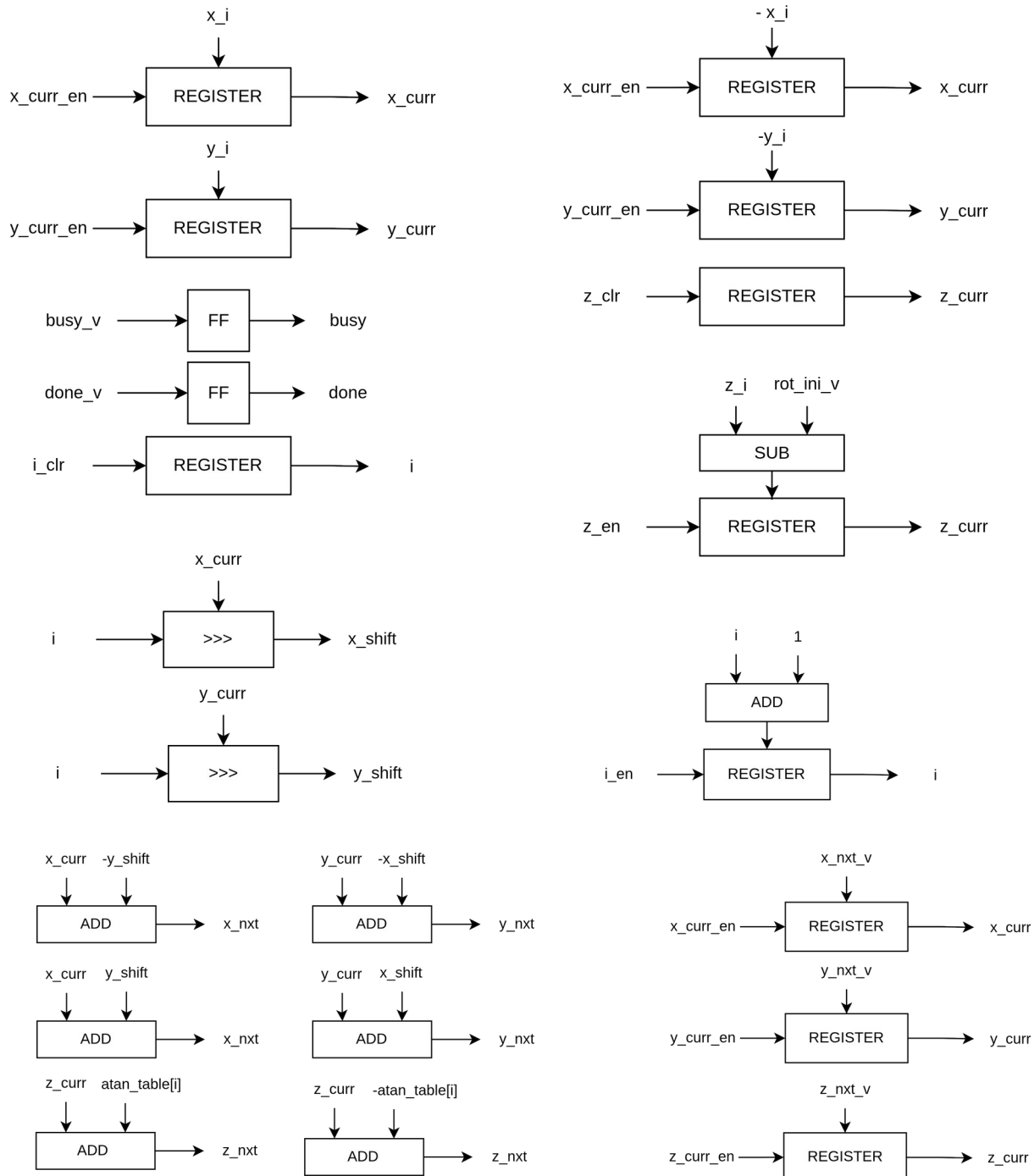
Diagrama ASM

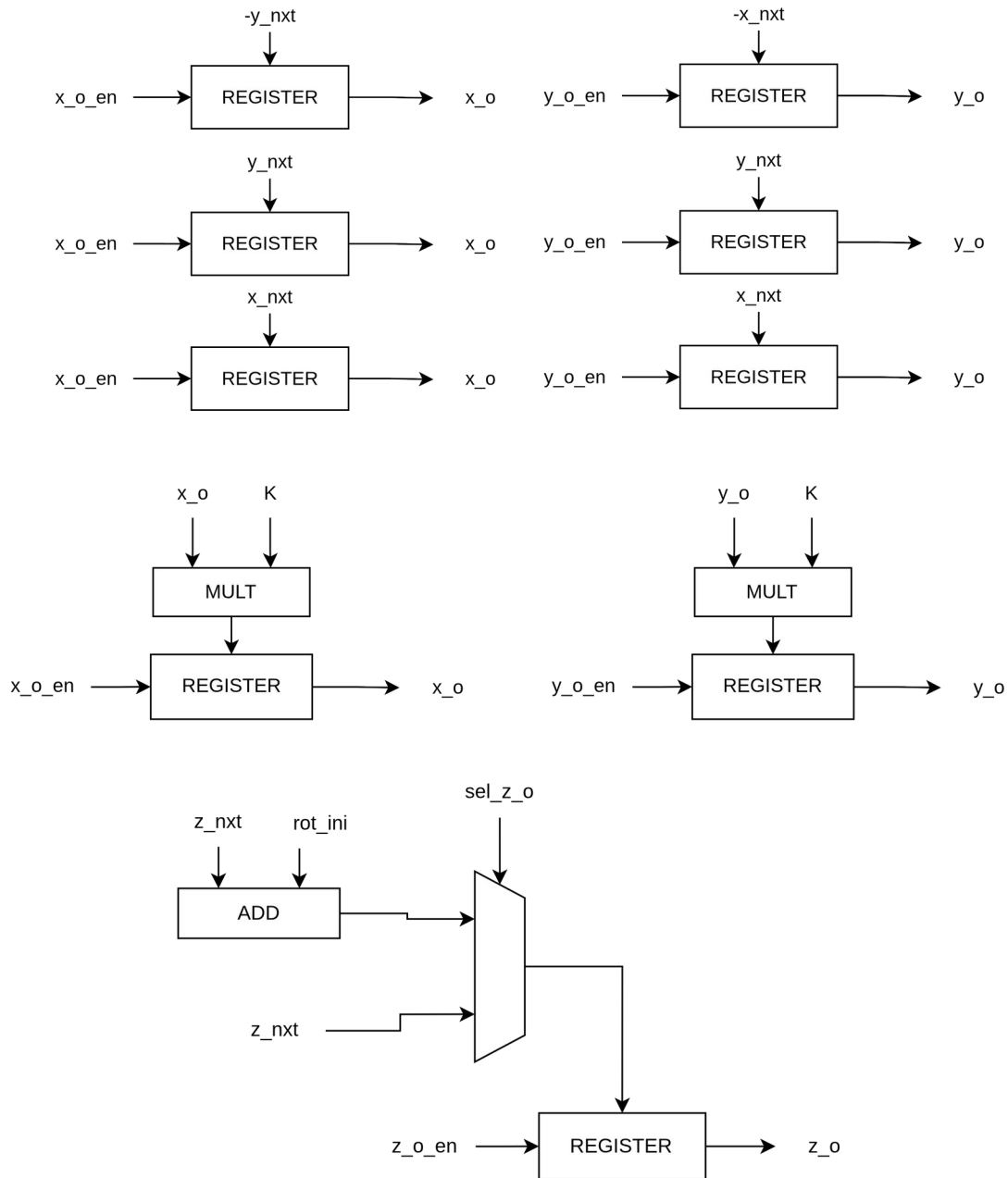


Podemos ver en el diagrama ASM combinación de instrucciones por estado, siguiendo la lógica de las tres principales etapas del algoritmo mencionadas anteriormente, con un estado extra previo a todos, que espera el inicio del proceso mediante la señal *start*.

Datapath

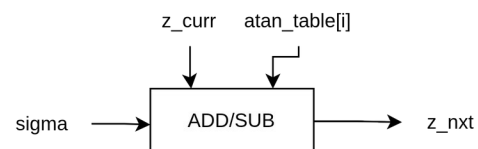
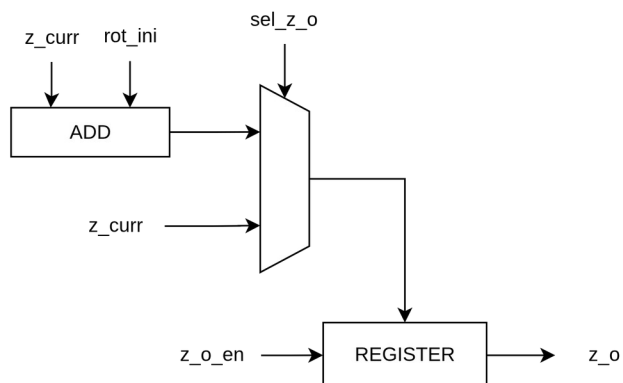
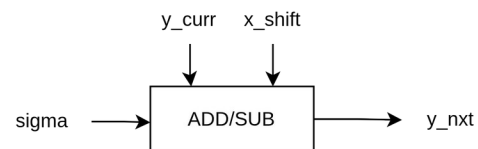
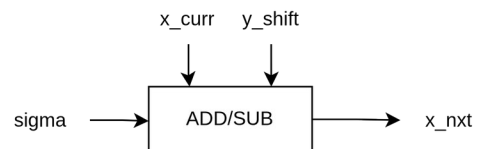
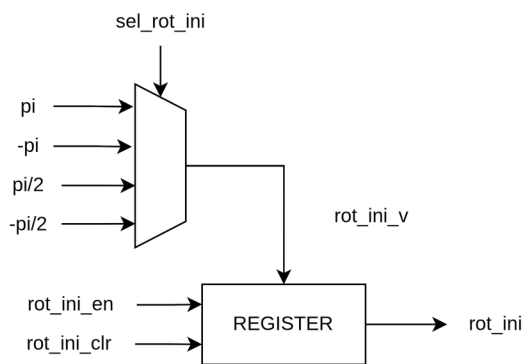
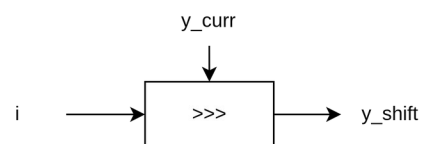
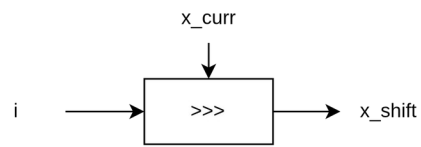
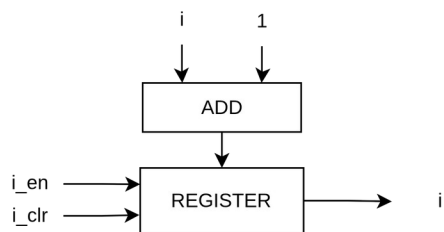
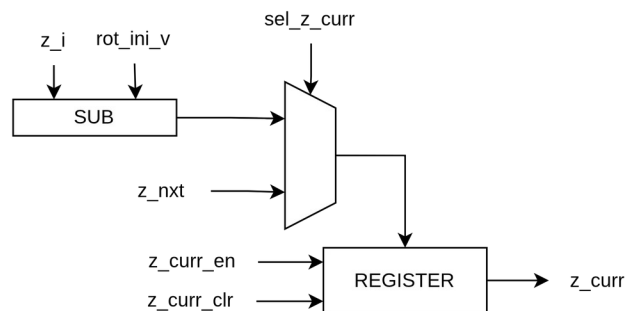
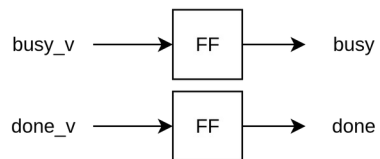
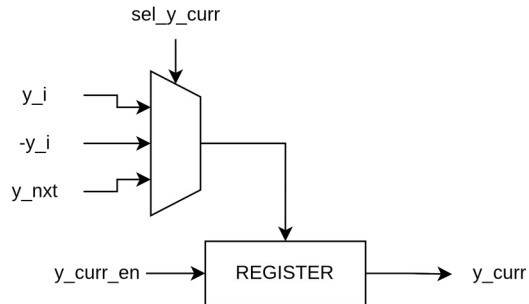
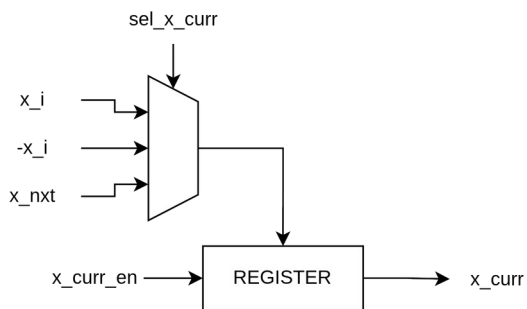
A continuación, los bloques necesarios en el datapath para cada una de las asignaciones realizadas en el ASM:

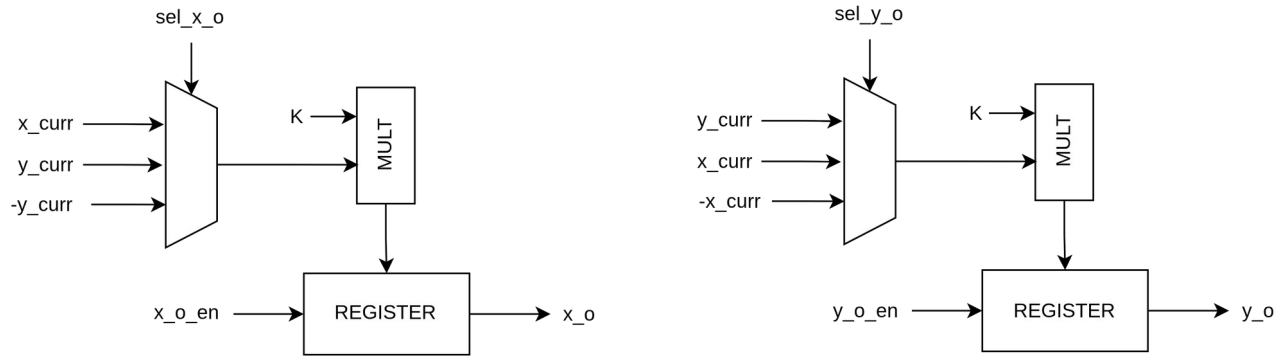




Optimizaci3n del datapath

Tras analizar los bloques de estado y su datapath, podemos ver redundancias y aspectos que podr3an ser optimizados. Los siguientes bloques representan las optimizaciones hechas:





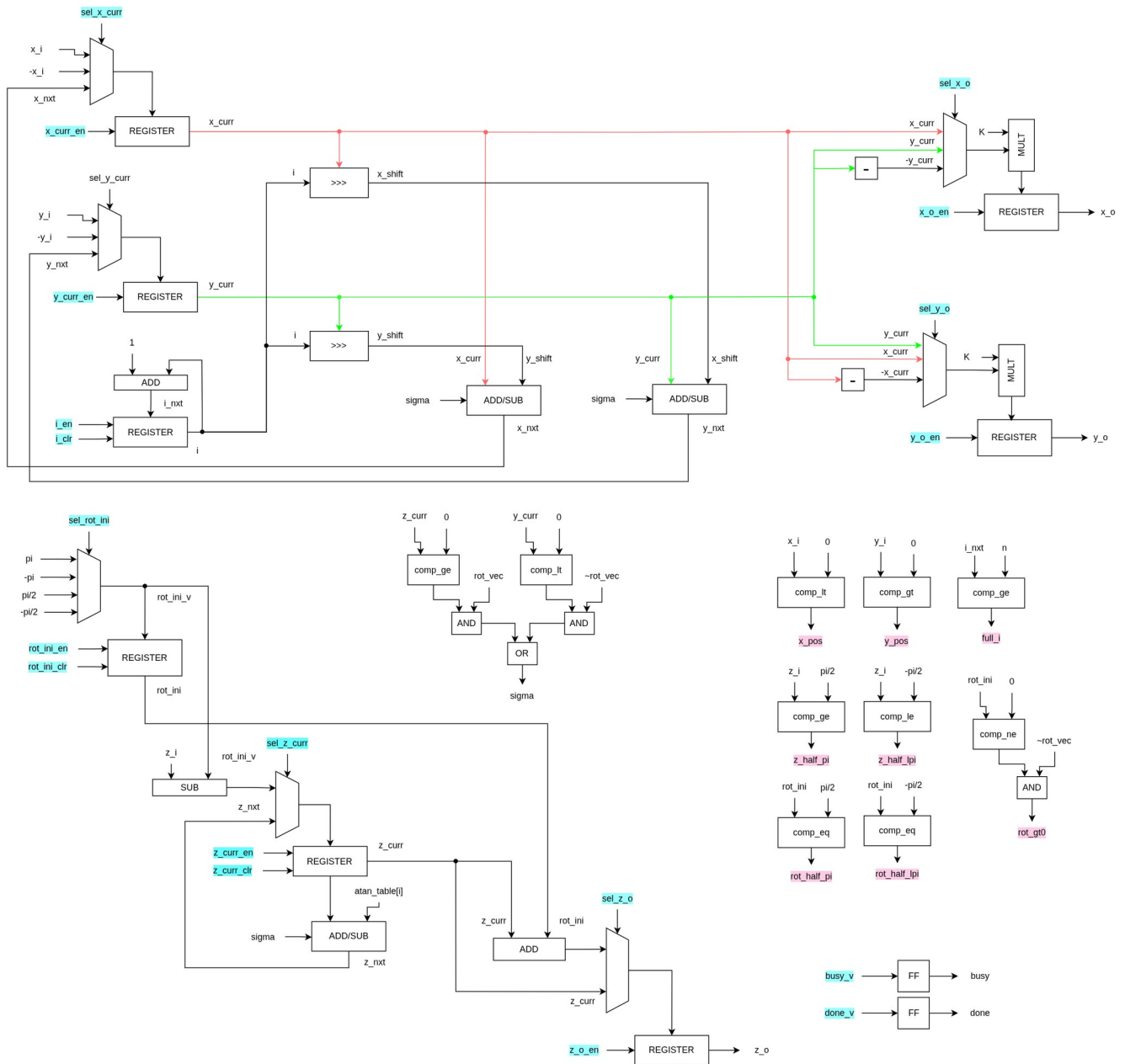
Podemos observar que muchas optimizaciones requieren de multiplexores. Es por eso que se infiere un módulo común a ellos llamado *mux_reg*, con el propósito de registrar la salida seleccionada del multiplexor. Se observa además que el bloque cuenta con una salida extra que se asigna a la salida combinacional, y no registrada, del multiplexor, dado que es necesario usar este dato dentro del mismo ciclo de reloj y no esperar a su registro. Este comportamiento se observa en el bloque optimizado de *rot_ini*.

Además de este bloque, hay uno muy parecido que realiza un paso extra anterior al registro, el cual es la multiplicación de la salida del multiplexor por el factor de escalamiento K . Este módulo es llamado *mux_mult_reg*.

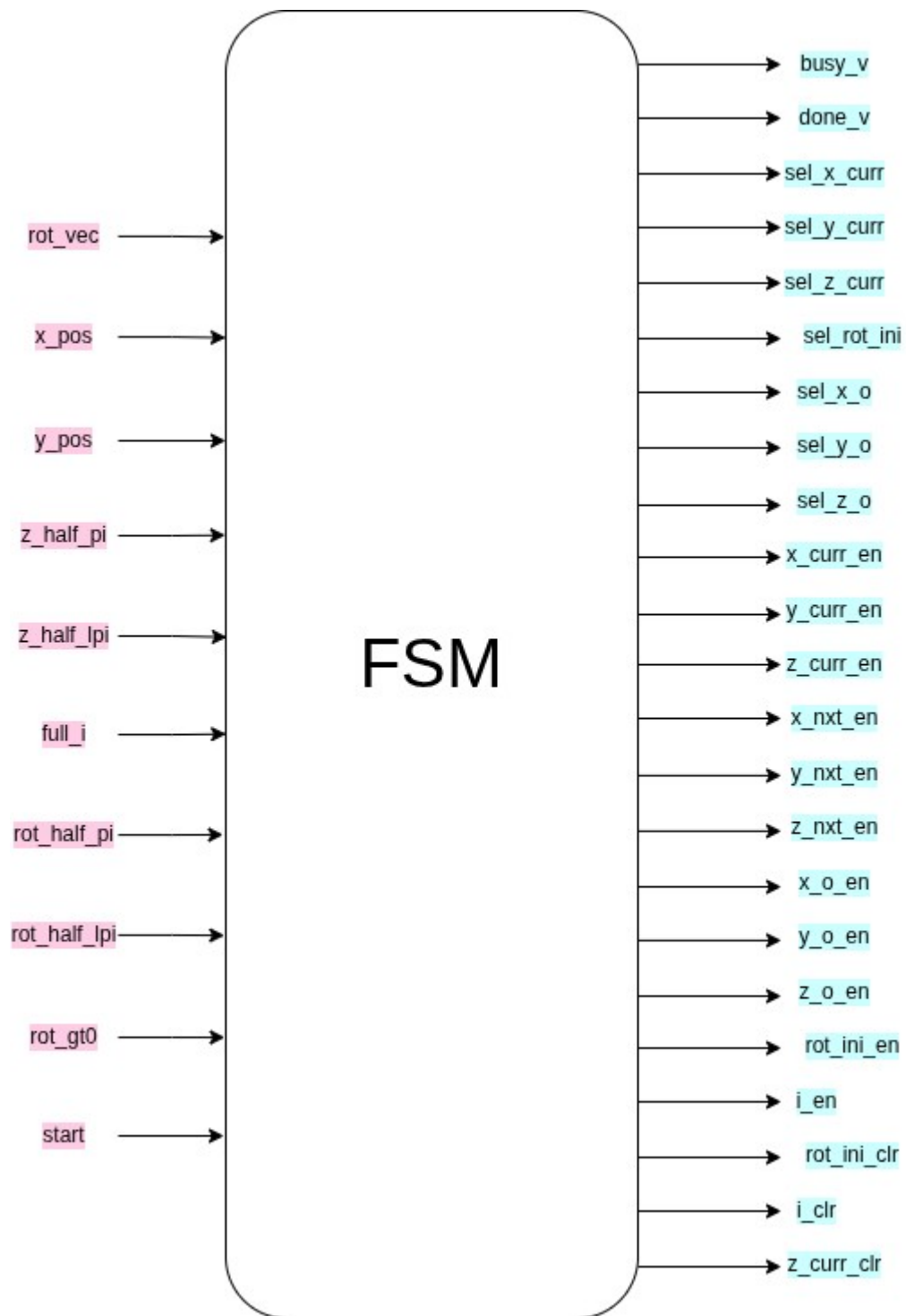
Por otra parte, existe un bloque optimizado llamado *add_sub*. Este módulo realiza una suma o una resta de sus dos entradas, controlado por la señal sigma, el cual es un resultado combinacional derivado de comparaciones.

Es de suma importancia mencionar que todas las optimizaciones de bloques, como de estados, se realizaron en función de tener una **menor latencia**, sin considerar que esto podría afectar malamente el tamaño del **critical path**, logrando una menor frecuencia máxima de uso para el circuito final.

Datapath final



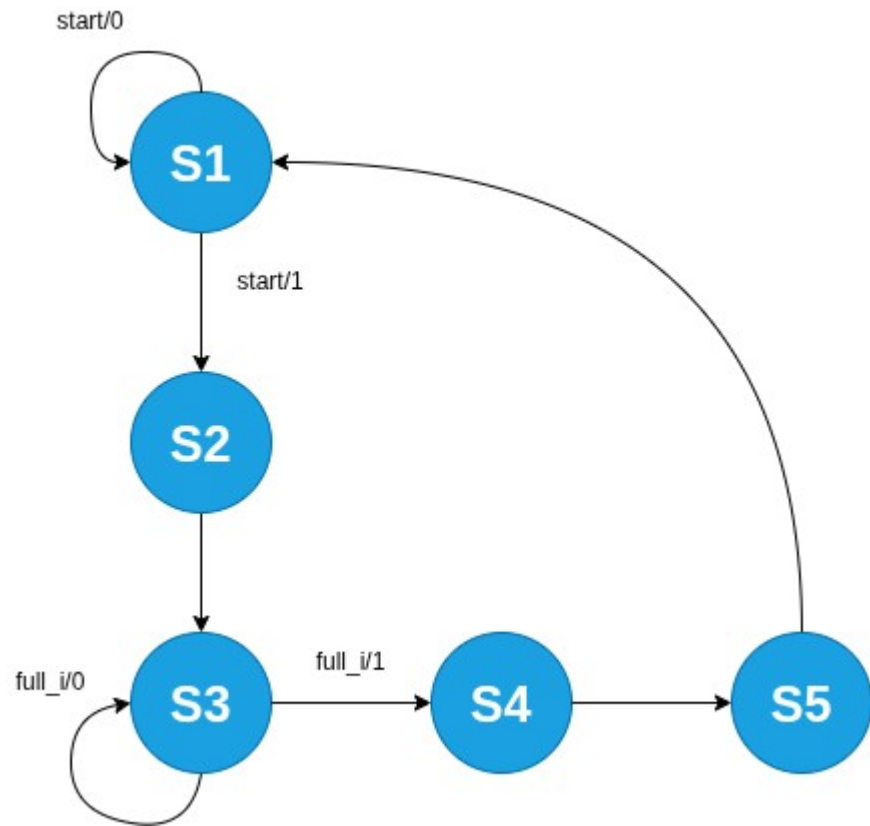
Máquina de estados

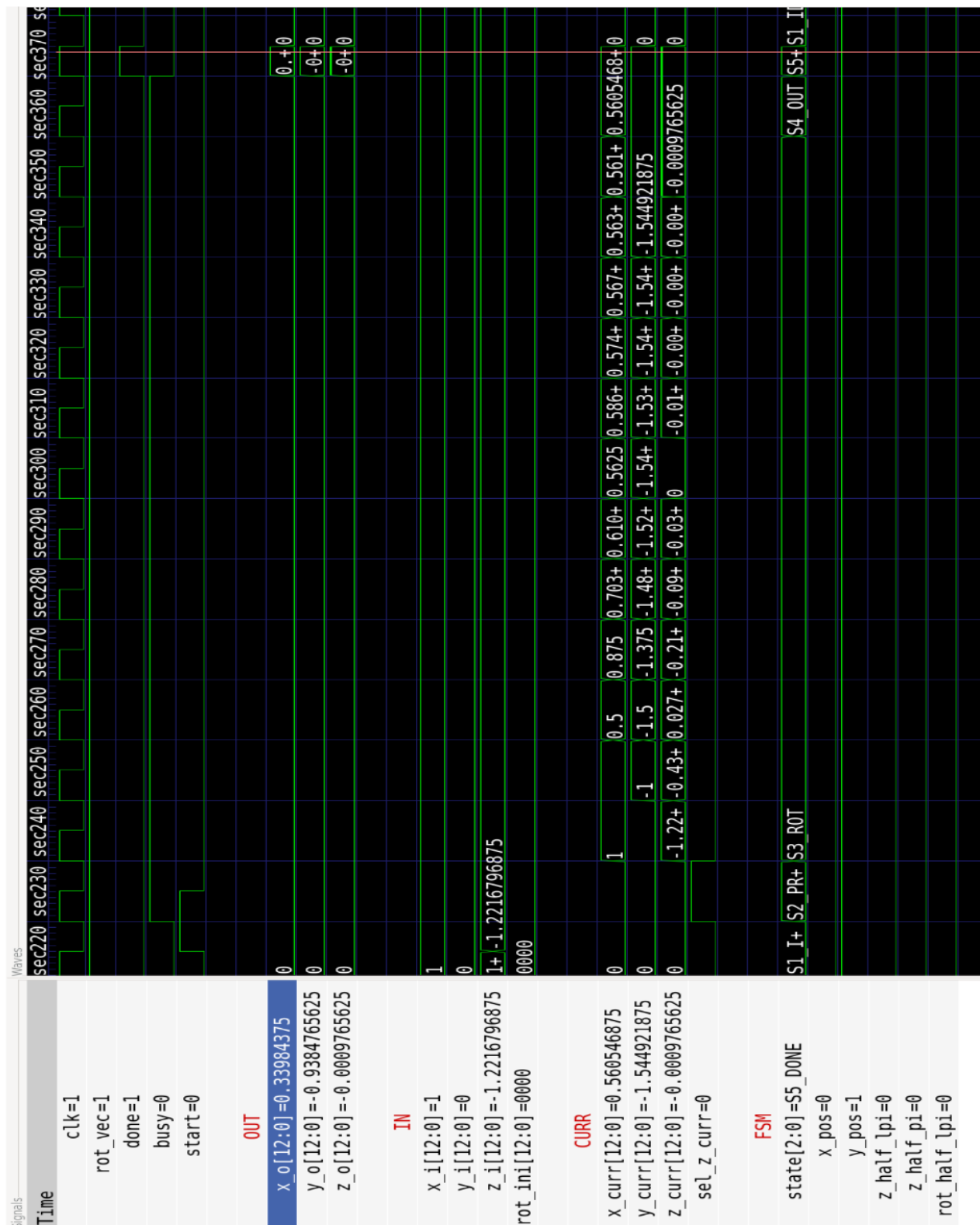


La máquina de estados finita define el comportamiento de sus salidas como una combinación de una máquina de estados **Moore**, porque hay salidas que dependen solamente del estado actual, y una tipo **Mealy**, porque hay salidas que dependen tanto del estado en que se encuentra como de las entradas de la máquina. Dicho de nuevo, esto es resultado de la optimización que busca reducir la latencia del algoritmo.

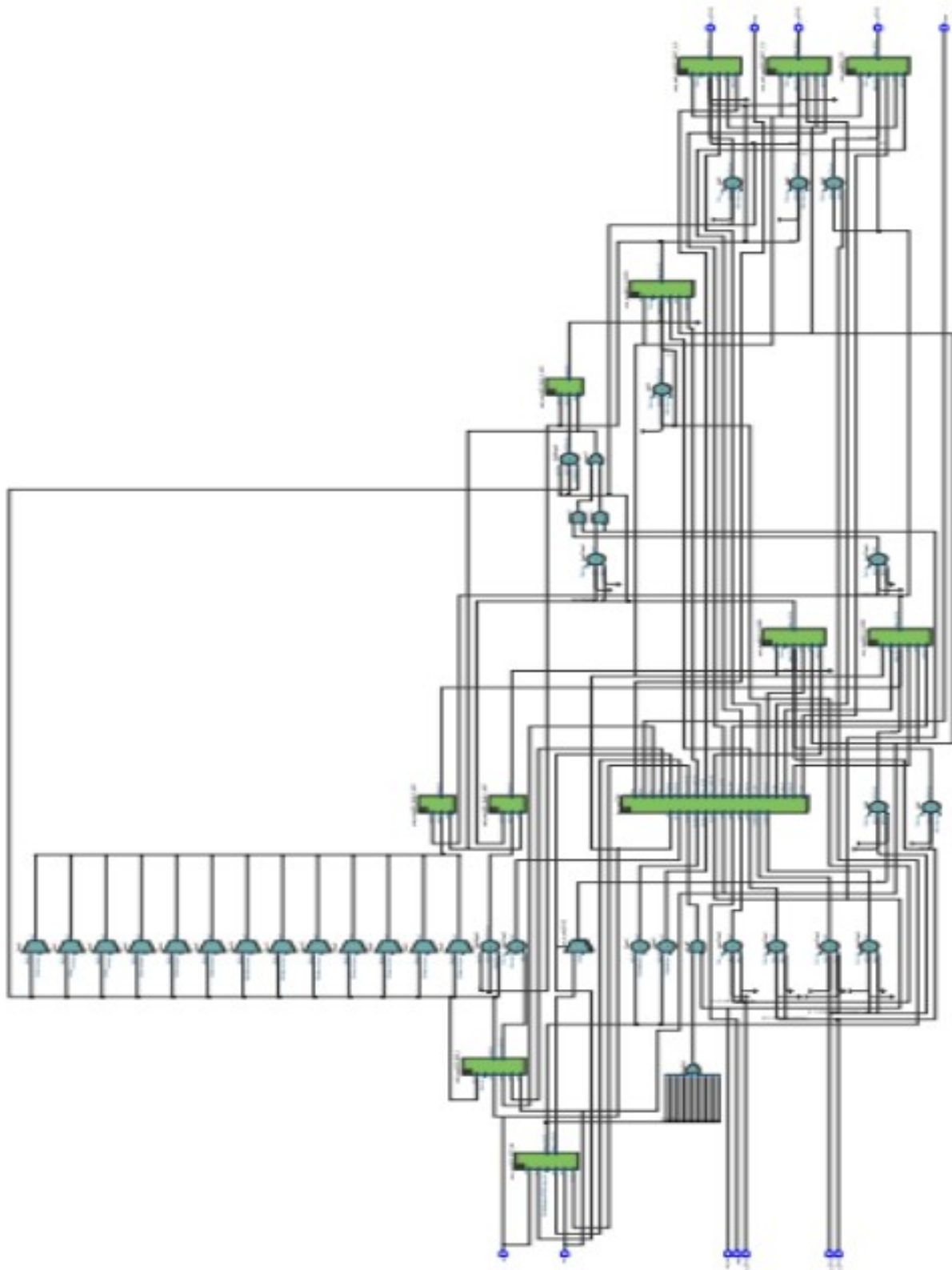
La siguiente tabla explica el comportamiento de las transiciones entre estados y los valores de salida para cada uno de ellos, donde las salidas tipo Mealy se representan como una función f que depende de valores de entrada $f(\dots)$.

FSM			
curr_state	input	next_state	output
S1	start	start/0 S1 start/1 S2	busy_v/0 done_v/0
S2	rot_vec x_pos y_pos z_pi z_lpi	S3	busy_v/1 sel_x_curr = $f(\text{rot_vec}, x_pos)$ sel_y_curr = $f(\text{rot_vec}, x_pos)$ sel_z_curr/1 sel_rot_ini = $f(\text{rot_vec}, x_pos, y_pos, z_pi, z_lpi)$ x_curr_en/1 y_curr_en/1 z_curr_en/1 z_curr_clr = $f(\text{rot_vec})$ rot_ini_en/1 rot_ini_clr = $f(\text{rot_vec}, x_pos, y_pos, z_pi, z_lpi)$ i_clr/1
S3	full_i	full_i/0 S3 full_i/1 S4	x_curr_en/1 y_curr_en/1 z_curr_en/1 i_en/1 sel_z_curr/0
S4	rot_vec rot_half_pi rot_half_lpi rot_gt0	S5	sel_x_o = $f(\text{rot_vec}, \text{rot_half_pi}, \text{rot_half_lpi})$ sel_y_o = $f(\text{rot_vec}, \text{rot_half_pi}, \text{rot_half_lpi})$ sel_z_o = $f(\text{rot_vec}, \text{rot_gt0})$ x_o_en/1 y_o_en/1 z_en/1
S5		S1	busy_v/0 done_v/1





Top Level RTL



La forma de onda se puede apreciar de mejor manera como imagen pdf en *simulation/wave.pdf*, o ejecutando *gtkwave simulation/wave.gtkw* para ver la forma de onda completa.

De la misma forma, para visualizar con mejoría el esquemático top level generado por Quartus, entrar a la herramienta directamente.

Todos los diagramas e imágenes mostradas hasta este momento se encuentran en las carpetas previamente mencionadas, o en la carpeta *drawings* en su defecto.

El número de pulsos de reloj que tarda el coprocesador en realizar el proceso de rotamiento o vectorización es de **14 ciclos de reloj** desde que la señal *start* es puesta en alto, hasta que *done* es puesto en alto. Esto concuerda con lo esperado por los 12 ciclos de las iteraciones referentes a las micro-rotaciones, el ciclo inicial para establecer rotación inicial y el ciclo final para sumar la rotación inicial a las salidas.

Resultado de recursos de simulación

Se obtuvieron resultados de una simulación en Quartus Prime para una Cyclone V.

	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	166
2		
3	▼ Combinational ALUT usage for logic	288
1	-- 7 input functions	0
2	-- 6 input functions	40
3	-- 5 input functions	104
4	-- 4 input functions	28
5	-- <=3 input functions	116
4		
5	Dedicated logic registers	108
6		
7	I/O pins	84
8		
9	Total DSP Blocks	2
10		
11	Maximum fan-out node	clk~input
12	Maximum fan-out	108
13	Total fan-out	1827
14	Average fan-out	3.23

	Fmax	Restricted Fmax	Clock Name	Note
1	125.71 MHz	125.71 MHz	clk	

Resultado de SQNR en simulación

Para evaluar la precisión del IP-core, el script SQNR.m lee los resultados producidos por la simulación RTL y los compara contra los valores teóricos calculados por el modelo de referencia en punto flotante de doble precisión (cordic.m). El SQNR se calcula como:

$$10 \cdot \log_{10} \left(\frac{\sum s_i^2}{\sum (s_i - \hat{s}_i)^2} \right)$$

donde s_i es el valor de referencia en punto flotante y $\hat{s}_i$ es la salida del DUT en punto fijo.

Los resultados obtenidos son los siguientes:

Modo	SQNR obtenido	Requisito	Margen
Rotación	66.49 dB	> 50 dB	+16.49 dB
Vectorización	66.76 dB	> 50 dB	+16.76 dB

Conclusión

Se diseñó e implementó un IP-core CORDIC en Verilog HDL siguiendo la metodología Top-Down, con soporte para los modos de rotación y vectorización, aritmética en punto fijo Q3.10 de 13 bits, y cobertura del rango completo de ángulos $(-\pi, \pi)$ mediante pre-rotación. El diseño fue validado en tres niveles: modelo de referencia en punto flotante (*cordic.m*), modelo en punto fijo (*cordic_fi.m*) y simulación RTL del coprocesador sintetizable.

En cuanto a precisión, el IP-core alcanzó un SQNR de **66.49 dB en modo rotación y 66.76 dB en modo vectorización**, superando el requisito de 50 dB con un margen de aproximadamente 16.5 dB en ambos casos. Este resultado valida la elección del formato Q3.10 con 12 iteraciones, donde el factor limitante de precisión es la resolución de la representación numérica y no el número de micro-rotaciones, dado que el error de truncamiento del algoritmo ($\sim 2^{-12}$) es menor que el paso de cuantización (2^{-10}).

En cuanto a latencia, el coprocesador completa cada operación en **14 ciclos de reloj** desde que la señal *start* es afirmada hasta que *done* es pulsado: 1 ciclo para establecer la pre-rotación, 12 ciclos de micro-rotaciones y 1 ciclo para aplicar la post-rotación y el factor de escala K.

La síntesis para una FPGA Cyclone V en Quartus Prime reportó una frecuencia máxima de **125.71 MHz**, con un consumo de 166 ALMs, 108 registros dedicados y 2 bloques DSP, correspondientes estos últimos a los multiplicadores del factor de escala CORDIC $K \approx 0.6073$. La frecuencia de operación alcanzada es consecuencia directa de la estrategia de diseño adoptada, que priorizó la reducción de latencia por encima de la minimización del critical path. Como trabajo futuro, la incorporación de segmentación por pipeline en las etapas de micro-rotación permitiría reducir el critical path y elevar la frecuencia máxima de operación, a costa de una mayor latencia absoluta por operación y un incremento moderado en el uso de registros.

Scripts

- **atan_table.m**: Genera los valores enteros representados en formato de punto fijo para la tabla de ángulos para el algoritmo cordic.
- **cordic_fi.m**: Algoritmo cordic en punto fijo.
- **cordic.m**: Algoritmo cordic en punto flotante.
- **cordic_test.m**: Pruebas del algoritmo cordic en punto fijo para establecer el formato de punto fijo que supere los 50 dB de SQNR.

- **validate_cordic.m:** Benchmark de validación de resultados del algoritmo cordic en punto flotante.
- **SQNR.m:** Cálculo del SQNR sobre los resultados leídos de la simulación RTL y los teóricos esperados.
- **gen_stimulus.py:** Genera los vectores de prueba que se utilizan en el la cama de pruebas.
- **run.sh:** Ejecuta todo el flujo de ejecución del diseño y el cálculo del SQNR.